General-Purpose Classical Reasoning System

AI Assignment Report

Natural-language knowledge base → internal representation → classical reasoning → True / False

Data note. The public datasets (RuleTaker, CounterBench and FOLIO) could not be downloaded when this report was generated. The experiments in Section 4 therefore consist solely of an engine-correctness check on synthetic data and must not be read as results on the public datasets. Running the notebook with internet access regenerates this report with the real figures.

1. Overview

This report describes a reasoning system that decides whether a query is logically entailed by a knowledge base written in natural language. The system follows the required pipeline: the text is first translated into an internal logical representation, and the final True/False decision is produced exclusively by classical reasoning algorithms. No generative AI model is used at any stage of the pipeline.

2. Chosen Representation

Every sentence of the knowledge base is converted by a deterministic, template-based parser into function-free Horn logic. A literal is a triple of a sign, a predicate and its arguments. A property sentence such as “Bob is young” becomes `young(bob)`, and a relational sentence such as “The cat chases the dog” becomes `chase(cat, dog)`, with verbs reduced to their base form. Rules are expressed as Horn clauses with a universally quantified variable: “If someone is smart and kind, then they are successful” becomes `smart(?x) ∧ kind(?x) → successful(?x)`. The same form is produced for generic statements such as “All red things are big” or “Big, red things are nice”. Conjunctive conclusions are split into separate rules, and a continuation such as “smart *and kind*” inherits the subject of the preceding condition.

Negation is supported under two semantics, and the better one is selected on development data. Under the **closed-world** semantics, a negated body literal means that the corresponding fact cannot be derived (negation as failure). Under the **open-world** semantics, a negated literal is an explicit, derivable fact, stored internally under the predicate `~p`. Sentences that match no template are retained as zero-argument propositional atoms and are counted as “unparsed”, so that parsing coverage can be reported honestly.

Two further internal forms are used when the data requires them. First-order formulas, such as the annotations that accompany FOLIO, are converted to clauses in conjunctive normal form with Skolem terms. Causal descriptions in the style of CounterBench are converted to

a Boolean structural causal model, in which “together cause” denotes a conjunction of parents.

3. Reasoning Algorithms

Forward chaining. The default engine is a semi-naive bottom-up procedure. In every round each rule is joined with the fact base so that at least one body literal matches a newly derived fact; unification is performed by pattern matching on ground tuples. Every derived fact records the rule instance that produced it, which allows complete proofs to be printed. The resulting model is cached per knowledge base, so several queries about the same theory require only one inference.

Negation as failure. For the closed-world semantics the engine uses the alternating fixpoint. An under-estimate and an over-estimate of the true facts are refined alternately until they no longer change. On stratified theories this yields the perfect model; otherwise it yields the well-founded model, and any undefined answer is returned as False.

Backward chaining. As a goal-directed alternative, the system implements SLD resolution with a loop check on ancestor goals, a depth limit, a step budget and negation as failure. It is used to compare efficiency with forward chaining.

Resolution. For first-order input the system eliminates the connectives $\rightarrow$, $\leftrightarrow$ and $\oplus$, converts to negation normal form, Skolemises existential quantifiers and distributes $\vee$ over $\wedge$ to obtain clauses. It then runs set-of-support resolution with unification (including the occurs check), factoring, tautology and duplicate removal, and smallest-clause-first selection under a time limit. A query is entailed (True) when the premises together with the negated conclusion are unsatisfiable, and contradicted (False) when the premises together with the conclusion are unsatisfiable; otherwise it is Uncertain, which is reported as False in the binary output.

Causal-model reasoning. Counterfactual queries are answered in three classical steps: abduction of the exogenous assignments consistent with the evidence, intervention on the variables that are set by the question, and prediction of the outcome in the modified model.

4. Experiments

Because the public datasets were unavailable, the system was verified on randomly generated RuleTaker-style theories containing facts, relations, negation and stratified rules. The labels of these theories come from an independent, grounding-based oracle, so agreement of 100% is expected and confirms that the parser and both chaining engines are correct. These figures say nothing about accuracy on the real benchmarks.

Dataset	Configuration	n	Accuracy (%)	Time (s)	ms / query
Synthetic (size 1)	forward chaining	600	100.00	0.05	0.08
Synthetic (size 1)	backward chaining	600	100.00	0.04	0.06
Synthetic (size 3)	forward chaining	600	100.00	0.16	0.27
Synthetic (size 3)	backward chaining	600	100.00	0.09	0.14
Synthetic (size 5)	forward chaining	600	100.00	0.36	0.60
Synthetic (size 5)	backward chaining	600	100.00	0.13	0.22

Table 1. Accuracy and inference time of each configuration.

The final system uses the closed-world negation semantics.

Forward chaining pays a one-off cost to build the model of each theory, whereas backward chaining is cheaper on large theories when only a single query is asked.

5. Examples of Reasoning Traces

The traces below are produced by the system itself. Each line shows a derived fact and the rule (quoted from the knowledge base) that justifies it, indented under the facts it depends on.

Assignment example. The example from the assignment is solved by two chained rule applications.

```
Query: Bob is successful.  →  True
    successful(bob)   ⇐ "If someone is smart and kind, then they are successful"
        smart(bob)   ⇐ "If someone is young, then they are smart"
            young(bob)   [given fact]
        kind(bob)   [given fact]
```

Negation as failure. Here the condition “not big” holds because no derivation of “big” exists.

```
Query: The cat is kind.  →  True
    kind(cat)   ⇐ "If something is not big and it is red then it is kind"
        red(cat)   [given fact]
        not big(cat)   [cannot be derived → NAF]
```

Failed query. A query that cannot be derived is answered False.

```
Query: The bald eagle is kind.  →  False
    kind(bald eagle): no derivation found → False
```

First-order resolution. A first-order entailment is proved by refutation.

```
Premises: ∀x (Dog(x) → Animal(x)) ; ∀x (Animal(x) → ∃y (Needs(x,y) ∧ Food(y))) ; Dog(rex)
Conclusion ∃y (Needs(rex,y) ∧ Food(y))  →  True
(refutation: clauses of the negated conclusion resolve with Skolemised premises to the empty clause)
```

6. Limitations

Coverage depends on the template parser. Free-form English, which is typical of the natural-language premises in FOLIO, is handled only through the first-order annotations when they are provided. Existential statements and rules with two distinct variables are not parsed, and the exact text format of CounterBench was matched with flexible patterns rather than verified against the released data. These cases are the main sources of expected errors, and extending the parser templates is the most direct way to improve accuracy.